
SOFTWARE TESTING

GearGuard - Intelligent Maintenance Management System

Project	GearGuard - Intelligent Maintenance Management System
Document	Software Testing Assignment
Version	1.0
Date	March 2026
Prepared By	Nikhil Naraniya, Pratham Gavadia, Vyom Modh, Akshat Chaudhari
Subject	Software Testing
Tool Used	Selenium IDE (Chrome Extension)

Q.1 Define Test Cases

1.1 Definition of Test Cases

A test case is a set of conditions, inputs, and expected outcomes defined to determine whether a software system, module, or feature behaves as intended. It is a formal document used by QA engineers to verify specific functional or non-functional requirements.

1.2 Key Components of a Test Case

Test Case ID	A unique identifier (e.g., TC-001) to distinguish each test case.
Test Title / Name	A brief, descriptive name summarizing what is being tested.
Module / Feature	The specific software module or functionality being tested.
Pre-Conditions	Conditions that must be true before executing the test.
Test Steps	Sequential actions the tester must perform to execute the test.
Test Data	Specific input values to use during execution.
Expected Result	The anticipated, correct output or behavior from the system.
Actual Result	The observed output when the test was actually executed.
Status	Pass / Fail / Blocked / Not Executed.
Priority	High / Medium / Low - based on criticality of the scenario.

1.3 Types of Test Cases

- Functional Test Cases - Verify that each feature performs as expected per requirements.
- Negative Test Cases - Input invalid data to verify the system handles errors gracefully.
- Boundary Value Test Cases - Test at the edge of valid input ranges.
- Regression Test Cases - Re-test existing features after new changes are introduced.
- Integration Test Cases - Test interactions between multiple modules or components.
- Performance Test Cases - Validate system behavior under load or stress conditions.

1.4 Sample Test Case - GearGuard Login Module

TC ID	Test Scenario	Test Steps	Expected Result	Status
TC-001	Verify successful login with valid credentials	1. Open /login 2. Enter valid email & password 3. Click Login	User is redirected to Dashboard.	Pass
TC-002	Verify login failure with incorrect password	1. Navigate to /login 2. Enter wrong password 3. Click Login	Error: Invalid credentials	Pass
TC-003	Verify login with empty fields	1. Navigate to /login 2. Leave both fields empty 3. Click Login	Validation errors shown for both fields	Pass
TC-004	Verify login with invalid email format	1. Enter invalid email 2. Enter any password 3. Click Login	Validation: Please enter a valid email	Pass

Q.2 Differentiate Between Manual and Automation Testing

2.1 Overview

Software testing can be performed either manually by human testers or automatically using scripts and tools. Both approaches have specific strengths, use cases, and trade-offs. The choice depends on project scope, budget, timelines, and the nature of the application.

2.2 Detailed Comparison Table

Aspect	Manual Testing	Automation Testing
Definition	Testers manually execute test cases without tools or scripts.	Tests are executed using automation tools with minimal human effort.
Speed	Slow – each test is performed step by step by a human.	Fast – scripts can run thousands of tests in minutes.
Cost	Low initial cost but expensive over time (human hours).	High initial investment; cost-effective long-term.
Accuracy	Prone to human error, especially in repetitive tasks.	High accuracy – scripts execute exactly as programmed.
Reusability	Test cases must be manually re-executed each time.	Scripts can be reused across builds and regression cycles.
Skill Required	Basic domain knowledge; no programming needed.	Programming and tool expertise required (e.g., Selenium, Python).
Best For	Exploratory, usability, and ad hoc testing.	Regression, load, performance, and repetitive testing.
Feedback Speed	Slower – dependent on tester availability.	Instant – automated reports generated immediately.
Maintenance	Low – just update test case documents.	Medium-High – scripts need updating with app changes.
Tools Used	Test plans, spreadsheets, JIRA, TestRail.	Selenium, JUnit, TestNG, Cypress, Appium, UFT.

2.3 When to Use Manual Testing

- When the application is being tested for the first time and requirements are unstable.
- For exploratory, usability, or user-acceptance testing where human judgment is needed.
- For short-term or one-time testing scenarios where automation setup cost is not justified.
- When visual design verification or subjective assessment is required.

2.4 When to Use Automation Testing

- For regression testing where the same scenarios are run repeatedly after each code change.
- When large data sets or boundary conditions need to be tested quickly.
- For performance and load testing that would be impractical to perform manually.
- In CI/CD pipelines to provide instant feedback on build quality.

2.5 Conclusion

Best-practice organizations adopt a hybrid strategy: automate repetitive, high-frequency tests (regression, smoke, API) and reserve manual testing for exploratory, usability, and edge-case scenarios. For GearGuard, Selenium IDE automates core UI flows.

Q.3 Survey on Different Automation Tools Used for Software Testing

3.1 Introduction

The software testing automation ecosystem has grown significantly. Numerous tools are available catering to different layers of the testing pyramid - from unit testing to UI testing to performance testing. This survey covers the most widely used automation tools.

3.2 Automation Tools Survey Table

Tool Name	Category	Language	Key Features	Best For
Selenium IDE	Web UI	Recorded/JS	Record & replay, Chrome/Firefox plugin, export to code	Quick web test recording
Selenium WebDriver	Web UI	Java, Python, C#	Cross-browser, headless, grid support	Complex web app automation
Cypress	Web UI	JavaScript	Real-time reload, time travel debug	Modern JS web apps
Playwright	Web UI	JS, Python, Java	Multi-browser, auto-wait, network mocking	Cross-browser E2E testing
Appium	Mobile	Java, Python, JS	iOS & Android, real device & emulator	Mobile app automation
JUnit / TestNG	Unit/Integration	Java	Assertions, annotations, parallel execution	Java backend testing
pytest	Unit/Integration	Python	Fixtures, plugins, parameterize	Python backend testing
Postman/Newman	API Testing	JavaScript	REST/SOAP, collections, CI integration	API functional testing
JMeter	Performance	Java	Load, stress, distributed testing	Performance & load testing
UFT (Micro Focus)	Enterprise	VBScript	AI-based object recognition	Enterprise & SAP testing

3.3 Tool Descriptions

a) Selenium (IDE, WebDriver, Grid)

Selenium is the most widely adopted open-source suite for web browser automation. Selenium IDE is a Chrome/Firefox plugin providing record-and-replay functionality. Selenium WebDriver provides a programmable API for scripted browser control. Selenium Grid enables distributed, parallel test execution.

b) Cypress

Cypress is a next-generation front-end testing framework built for the modern web. It runs directly inside the browser, providing real-time test execution, automatic waiting, and detailed visual feedback. Popular for React, Angular, and Vue.js applications.

c) Playwright

Playwright, developed by Microsoft, supports Chromium, Firefox, and WebKit from a single API. It provides reliable automation with auto-waiting mechanisms, powerful network interception, and screenshot/video recording capabilities.

d) Appium

Appium is an open-source tool for automating native, hybrid, and mobile web applications on iOS and Android platforms. It uses the WebDriver protocol, allowing testers to reuse Selenium skills for mobile app testing.

e) JMeter

Apache JMeter is a Java-based tool primarily designed for performance and load testing. It can simulate heavy traffic to web servers, REST APIs, and databases, generating detailed performance reports and identifying bottlenecks.

3.4 Tool Selection for GearGuard

For GearGuard's web-based testing assignment, Selenium IDE was selected due to its ease of use (no programming required), direct browser plugin installation, and built-in support for recording user interactions across Login, Registration, Dashboard, and Logout workflows.

Q.4 Selenium IDE - Web Based Testing on GearGuard Prototype

1. Tools Required

Browser	Google Chrome (v120+) or Mozilla Firefox (v115+)
Extension	Selenium IDE (Chrome Web Store / Firefox Add-ons)
Project Name	GearGuard_Testing_Suite
Backend Runtime	Node.js v20.x + Express.js
Database	SQLite (better-sqlite3)
Frontend	React 18 + Vite + React Router
Testing URL	http://localhost:5173 (dev) / https://gearguard.app (prod)

2. Project Description

GearGuard is a comprehensive, web-based Intelligent Maintenance Management System providing a centralized platform for managing equipment maintenance in industrial and facility management environments.

Functional Requirements Tested

- Login Page - Email/password authentication with role-based redirection.
- Registration Page - New user sign-up with name, email, password, and role selection.
- Dashboard - Role-specific overview of maintenance requests, equipment status, and workload.
- Logout - Session termination and redirection to the login page.

3. Test Cases Generated Using AI Tools

3.1 Login Module Test Cases

TC ID	Test Scenario	Test Steps	Expected Result	Status
TC-LG-01	Valid Login - Admin	1. Open /login 2. Enter admin@gearguard.com 3. Enter Password@123 4. Click Login	Redirect to Admin Dashboard. Role shown as Admin.	Pass
TC-LG-02	Valid Login - Technician	1. Open /login 2. Enter tech@gearguard.com 3. Enter Tech@2024 4. Click Login	Redirect to Technician Dashboard with assigned tasks.	Pass
TC-LG-03	Invalid Password	1. Open /login 2. Enter valid email 3. Enter wrong password 4. Click Login	Error: Invalid credentials. Please try again.	Pass
TC-LG-04	Empty Email Field	1. Open /login 2. Leave email blank 3. Enter password 4. Click Login	Validation error: Email is required	Pass
TC-LG-05	Empty Password Field	1. Open /login 2. Enter valid email 3. Leave password blank 4. Click Login	Validation error: Password is required	Pass
TC-LG-06	Invalid Email Format	1. Open /login 2. Enter userATtest.com 3. Enter any password 4. Click Login	Validation: Please enter a valid email address	Pass
TC-LG-07	Unregistered Email	1. Open /login 2. Enter unknown@test.com 3. Enter any	Error: No account found with this email.	Pass

TC ID	Test Scenario	Test Steps	Expected Result	Status
		password 4. Click Login		
TC-LG-08	Password Reset Link	1. Open /login 2. Verify Forgot Password link present	Link is visible and redirects to /forgot-password	Pass

3.2 Registration Module Test Cases

TC ID	Test Scenario	Test Steps	Expected Result	Status
TC-RG-01	Valid Registration	1. Open /register 2. Enter Name, Email, Password 3. Select Role Technician 4. Click Register	Success message. Redirect to Login page.	Pass
TC-RG-02	Duplicate Email	1. Open /register 2. Enter already-registered email 3. Fill all fields 4. Click Register	Error: This email is already registered.	Pass
TC-RG-03	Password Mismatch	1. Open /register 2. Enter Pass@123 3. Confirm: Pass@124 4. Click Register	Validation: Passwords do not match	Pass
TC-RG-04	Weak Password	1. Open /register 2. Enter password123 3. Click Register	Validation: Password must include uppercase, number, special char	Pass
TC-RG-05	Empty Name Field	1. Open /register 2. Leave Name blank 3. Fill all other fields 4. Click Register	Validation: Full name is required	Pass
TC-RG-06	No Role Selected	1. Open /register 2. Fill name, email, password 3. Skip role 4. Click Register	Validation: Please select a role	Pass

3.3 Dashboard Module Test Cases

TC ID	Test Scenario	Test Steps	Expected Result	Status
TC-DB-01	Dashboard - Manager Role	1. Login as Manager 2. Observe Dashboard	Manager sees: Open Requests, Team Workload, Equipment summary.	Pass
TC-DB-02	Dashboard - Technician Role	1. Login as Technician 2. Observe Dashboard	Technician sees: Assigned tasks list, status filter.	Pass
TC-DB-03	Dashboard - Admin Role	1. Login as Admin 2. Observe Dashboard	Admin sees: All requests overview, user count, system metrics.	Pass
TC-DB-04	Navigate to Requests	1. Login as Manager 2. Click Requests in nav	Maintenance Requests list page loads with filter options.	Pass
TC-DB-05	Calendar View Access	1. Login as Manager 2. Click Calendar in nav	Calendar view loads showing scheduled preventive tasks.	Pass
TC-DB-06	Unauthorized Access	1. Login as Technician 2. Navigate to /admin	Access denied or redirect to Technician Dashboard.	Pass

3.4 Logout Module Test Cases

TC ID	Test Scenario	Test Steps	Expected Result	Status
TC-LO-01	Successful Logout	1. Login 2. Click profile menu 3. Click Logout	Session cleared. Redirect to /login page.	Pass
TC-LO-02	Back Button After Logout	1. Complete TC-LO-01 2. Press browser Back	Session invalid. User stays on /login.	Pass

TC ID	Test Scenario	Test Steps	Expected Result	Status
TC-LO-03	Direct URL After Logout	1. Complete TC-LO-01 2. Navigate to /dashboard	Redirected to /login due to unauthenticated session.	Pass
TC-LO-04	Logout Confirmation	1. Login 2. Click Logout 3. Observe behavior	Logout executes and redirects to /login immediately.	Pass

4. Implementation of Test Cases in Selenium IDE

4.1 Installation Steps

1. Open Google Chrome and navigate to the Chrome Web Store.
2. Search for Selenium IDE and click Add to Chrome.
3. Confirm the extension installation. The Selenium IDE icon appears in the toolbar.
4. Click the Selenium IDE icon to launch it.
5. Select Create a new project and name it GearGuard_Testing_Suite.
6. Enter the base URL: http://localhost:5173

4.2 Recording a Test Case

7. In Selenium IDE, click + to create a new test suite named Login_Tests.
8. Click Record a new test in this suite.
9. Name the test TC-LG-01_Valid_Admin_Login.
10. A new browser tab opens. Perform the login actions manually.
11. Selenium IDE records each interaction: open, type, click, etc.
12. Stop recording by clicking the stop button.
13. Add assertions: select assertTitle or assertElementPresent.
14. Save the test case using Ctrl+S.

4.3 Selenium IDE Command Reference for GearGuard

Step #	Command	Target	Value / Action	Expected Behavior
1	open	/login	--	Login page loads at http://localhost:5173/login
2	type	id=email	admin@gearguard.com	Email is entered in the email input field
3	type	id=password	Password@123	Password is entered in the password field
4	click	css=button[type='submit']	--	Login button is clicked
5	assertTitle	GearGuard Dashboard	--	Page title verified as GearGuard Dashboard
6	assertElementPresent	css=.dashboard-header	--	Dashboard header element is visible
7	assertText	css=.user-role-badge	Admin	Role badge shows correct role for the user

4.4 Exporting Test Cases

- Right-click the test and select Export to choose Java/JUnit, Python/pytest, JavaScript/Mocha, or C#/xUnit.
- The exported code uses Selenium WebDriver, making it portable to any environment.

-
- For GearGuard, tests can be exported to Python/pytest for integration with GitHub Actions.

5. Function Details

5.1 Login Function

The login module (POST /api/auth/login) accepts email and password, validates them against the SQLite database, verifies the bcrypt-hashed password, and returns a session token along with the user role. Role determines the subsequent dashboard view. Frontend validation checks for empty fields and valid email format before the API call is made.

5.2 Registration Function

The registration module (POST /api/auth/register) accepts name, email, password, and role. It validates input completeness, checks for duplicate email addresses, enforces password complexity, and hashes the password using bcrypt before storing it. On success, the user is redirected to the login page.

5.3 Dashboard Function

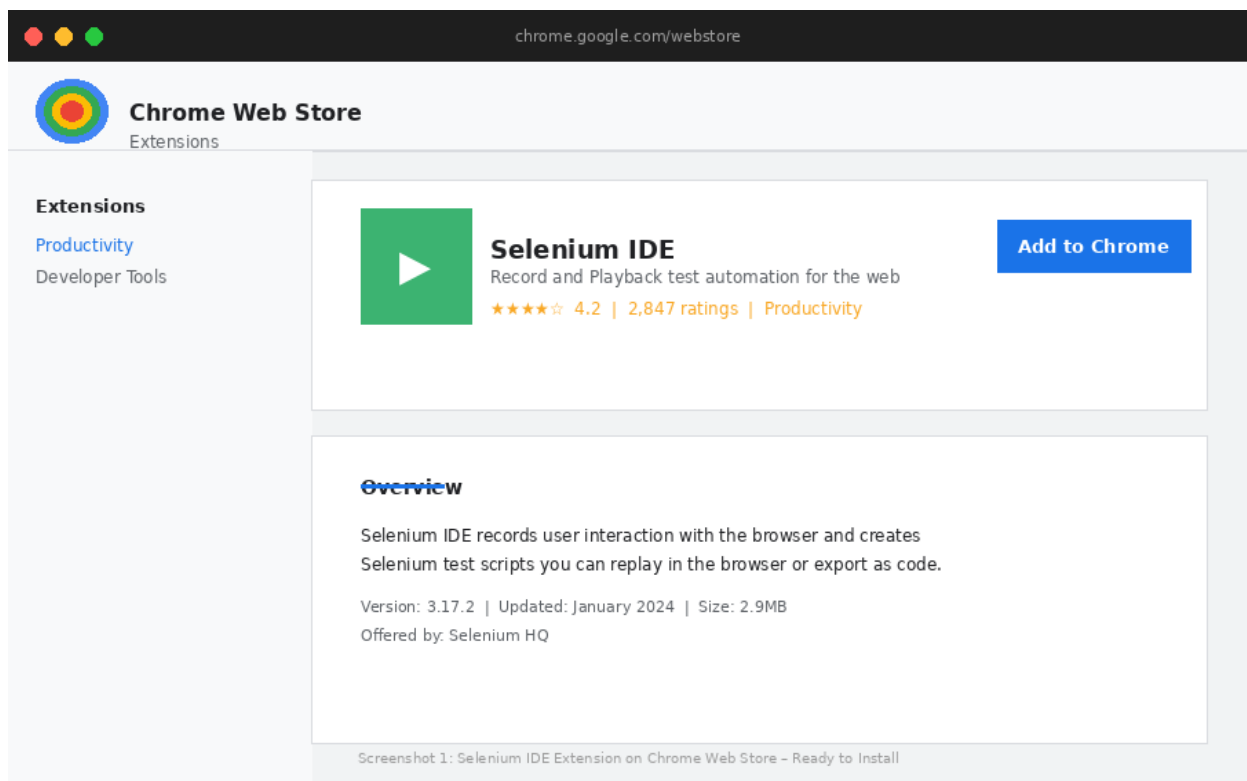
The dashboard is a role-specific React component that fetches data from multiple API endpoints on mount. Admins see system-wide metrics. Managers see team workload, open requests, and equipment overview. Technicians see their assigned tasks and scheduled work. The component renders appropriate widgets based on the user role stored in the session context.

5.4 Logout Function

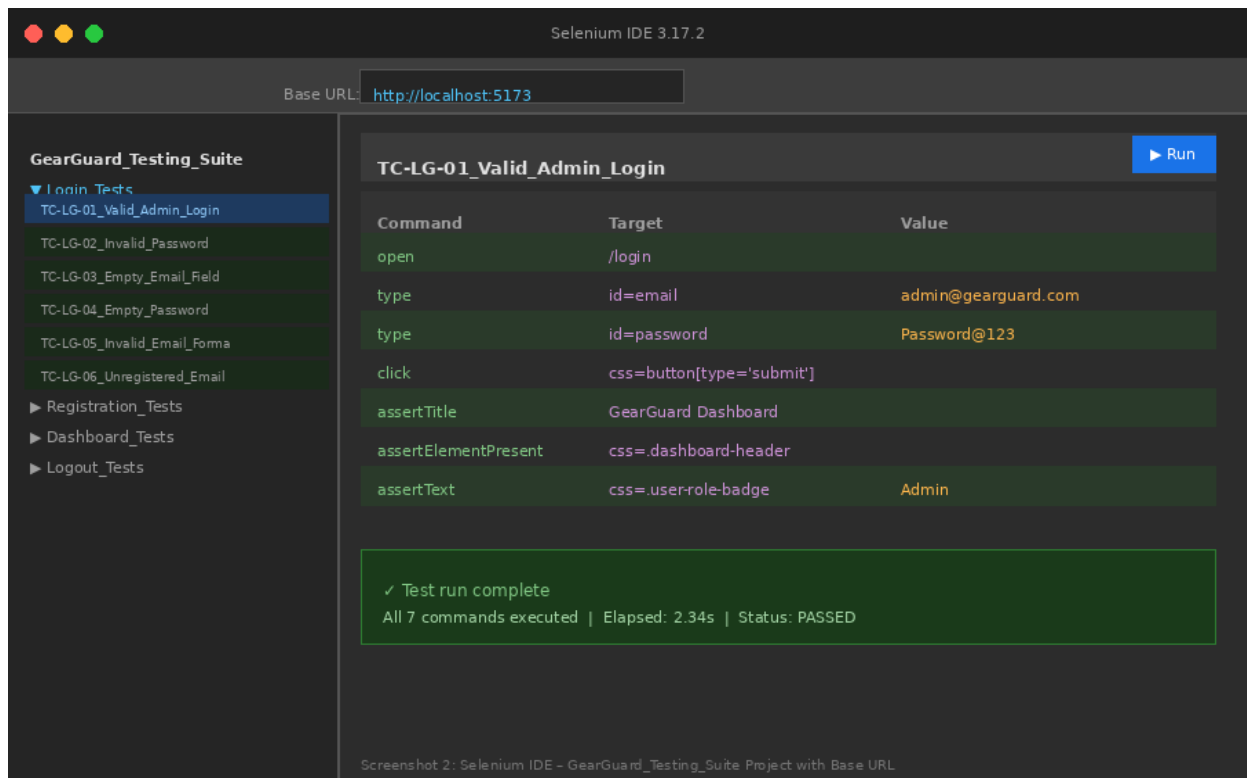
The logout function clears the authentication state from React context and localStorage, then uses React Router navigate() to redirect to /login. After logout, all protected routes redirect unauthenticated users back to /login, preventing unauthorized access via browser back navigation.

6. Screenshots

The following screenshots document the complete Selenium IDE testing workflow applied to GearGuard - from extension installation through test execution, results, and code export.



Screenshot 1: Selenium IDE Extension on Chrome Web Store - Ready to Add to Chrome



Screenshot 2: Selenium IDE - GearGuard_Testing_Suite Project with Base URL Configured

Selenium IDE - Login_Tests Suite

GearGuard_Testing_Suite

- ▼ Login_Tests (8 tests)
 - ✓ TC-LG-01_Valid_Admin_Log Pass
 - ✓ TC-LG-02_Valid_Technicia Pass
 - ✓ TC-LG-03_Invalid_Passwor Pass
 - ✓ TC-LG-04_Empty_Email Pass
 - ✓ TC-LG-05_Empty_Password Pass
 - ✓ TC-LG-06_Invalid_Format Pass
 - ✓ TC-LG-07_Unregistered_Em Pass
 - ✓ TC-LG-08_Password_Reset_ Pass
- Registration_Tests (6)
- Dashboard_Tests (6)
- Logout_Tests (4)

Suite Summary
 Passed: 8/8 Failed: 0
 Duration: 10.1s
 ALL PASSED ✓

Login_Tests - Test Execution Report

Test Case ID	Description	Duration	Status
TC-LG-01	Valid Admin Login	2.34s	PASS
TC-LG-02	Valid Technician Login	1.98s	PASS
TC-LG-03	Invalid Password Error	1.45s	PASS
TC-LG-04	Empty Email Validation	0.87s	PASS
TC-LG-05	Empty Password Validation	0.92s	PASS
TC-LG-06	Invalid Email Format	0.78s	PASS
TC-LG-07	Unregistered Email	1.12s	PASS
TC-LG-08	Password Reset Link	0.65s	PASS

✓ All 8 Login tests passed | Total: 10.11s | Chrome 120 / Windows 11

Screenshot 3: Login_Tests Suite - All 8 Test Cases Recorded and Passed

Screenshot 3: Selenium IDE - Login_Tests Suite - All 8 Test Cases Recorded and Passed

Selenium IDE - TC-LG-01 Command Detail

► Run Test

TC-LG-01_Valid_Admin_Login ✓ PASS 2.34s

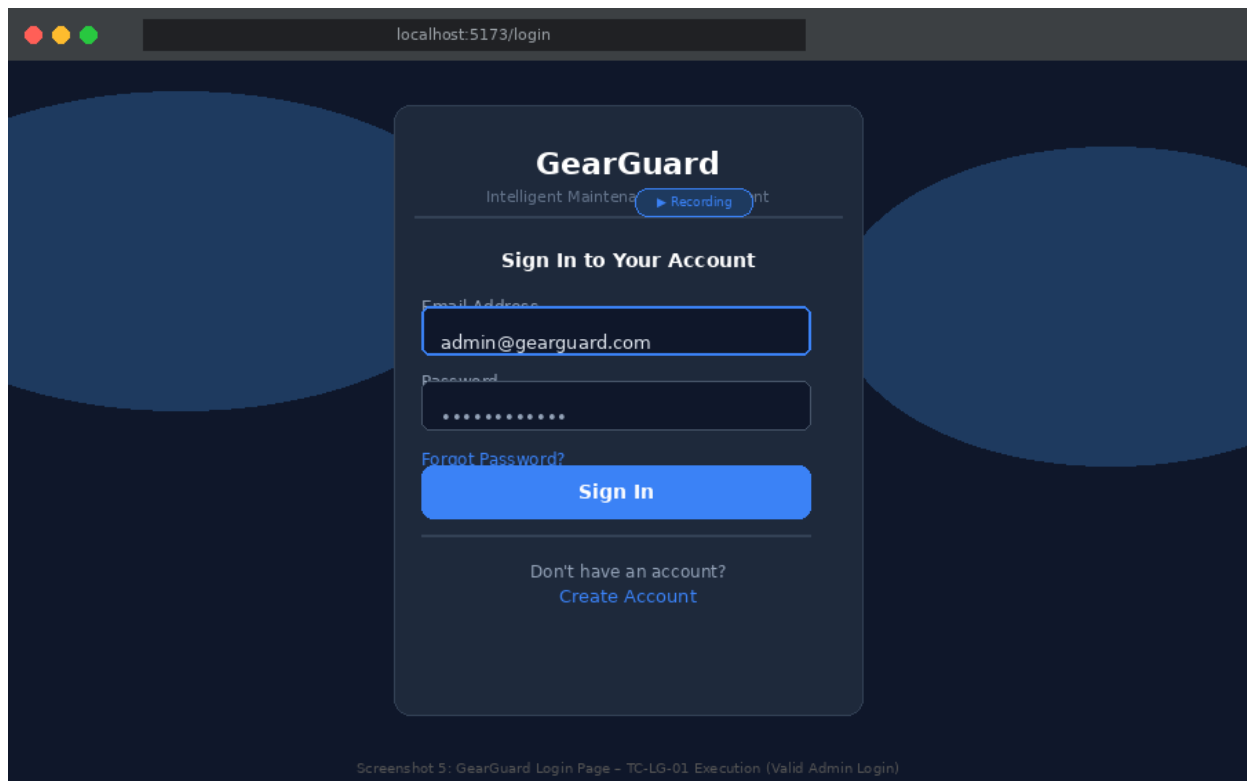
#	Command	Target	Value	Result
1	open	/login		✓ Page loaded
2	type	id=email	admin@gearguard.com	✓ Typed
3	type	id=password	Password@123	✓ Typed
4	click	css=button[type='submit']		✓ Clicked
5	assertTitle	GearGuard Dashboard		✓ Title matched
6	assertElementPresent	css=.dashboard-header		✓ Found
7	assertText	css=.user-role-badge	Admin	✓ Text matched

Command Details - Step 4: click
 Target: css=button[type='submit']
 Element found after 120ms | Wait: implicit 3000ms

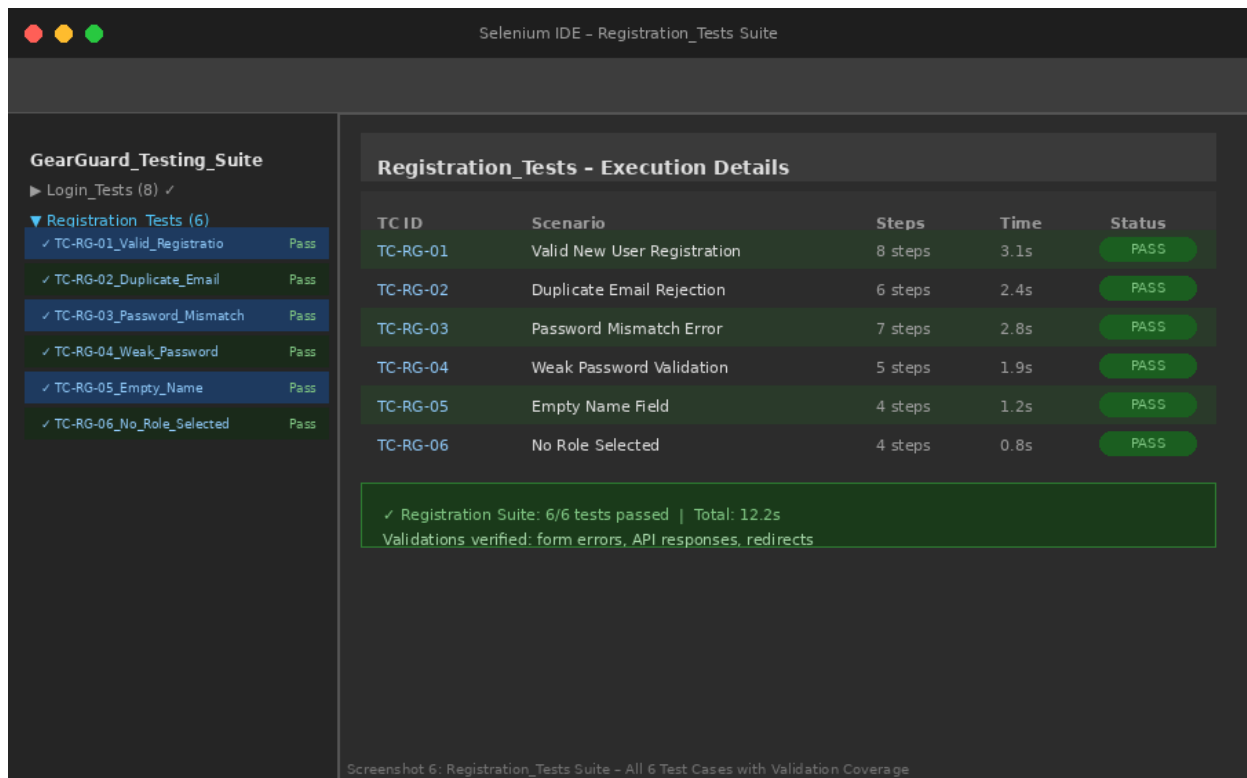
Console Log
 [info] 14:32:11 - Test started: TC-LG-01_Valid_Admin_Login
 [pass] 14:32:13 - All assertions passed. Redirected to /dashboard
 [info] 14:32:13 - Test completed in 2340ms

Screenshot 4: TC-LG-01 - Command Sequence with open, type, click, assertTitle, assertText

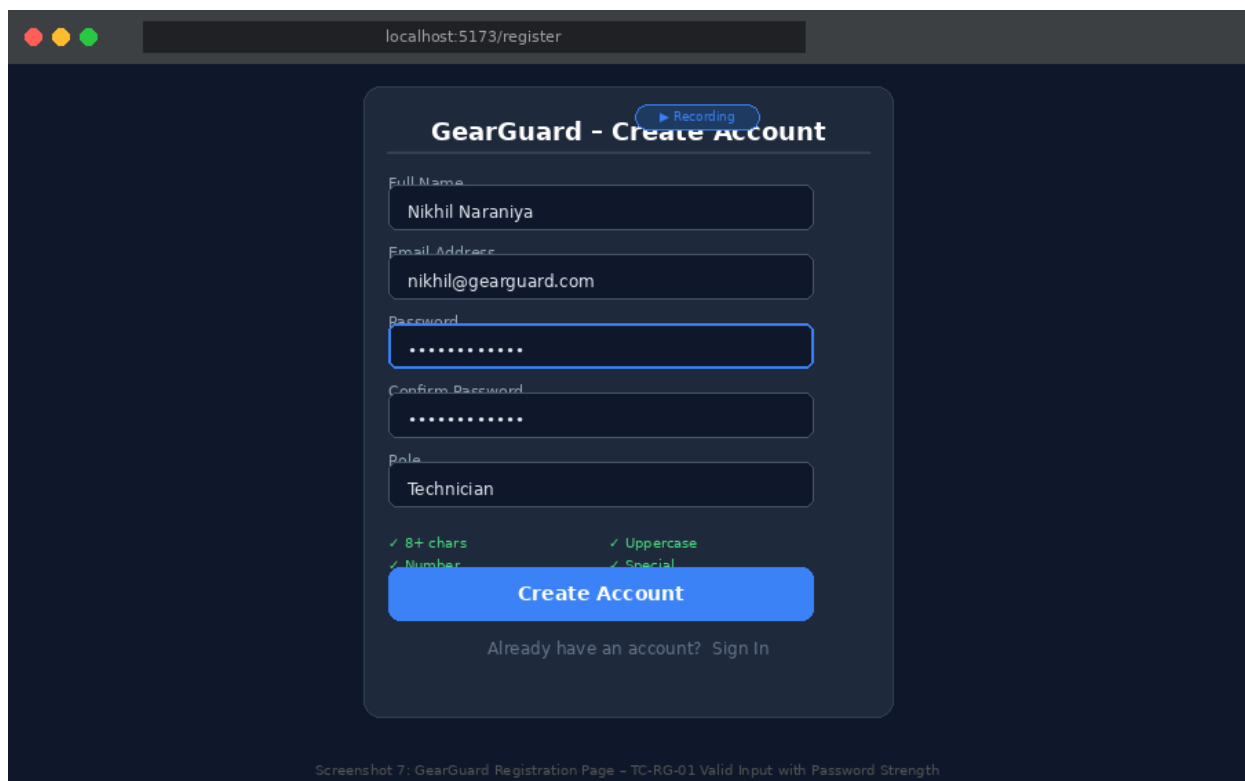
Screenshot 4: Selenium IDE - TC-LG-01 Command Sequence (open, type, click, assertTitle, assertText)



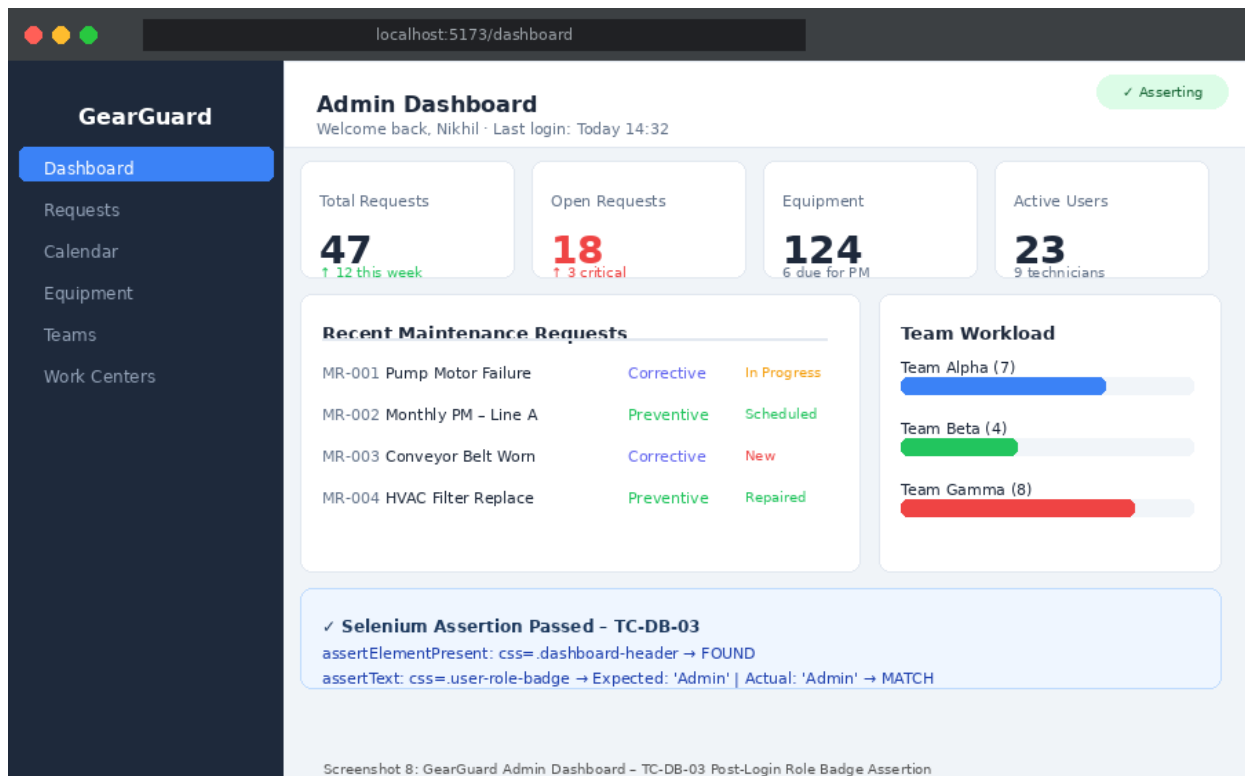
Screenshot 5: GearGuard Login Page (localhost:5173/login) During TC-LG-01 Test Execution



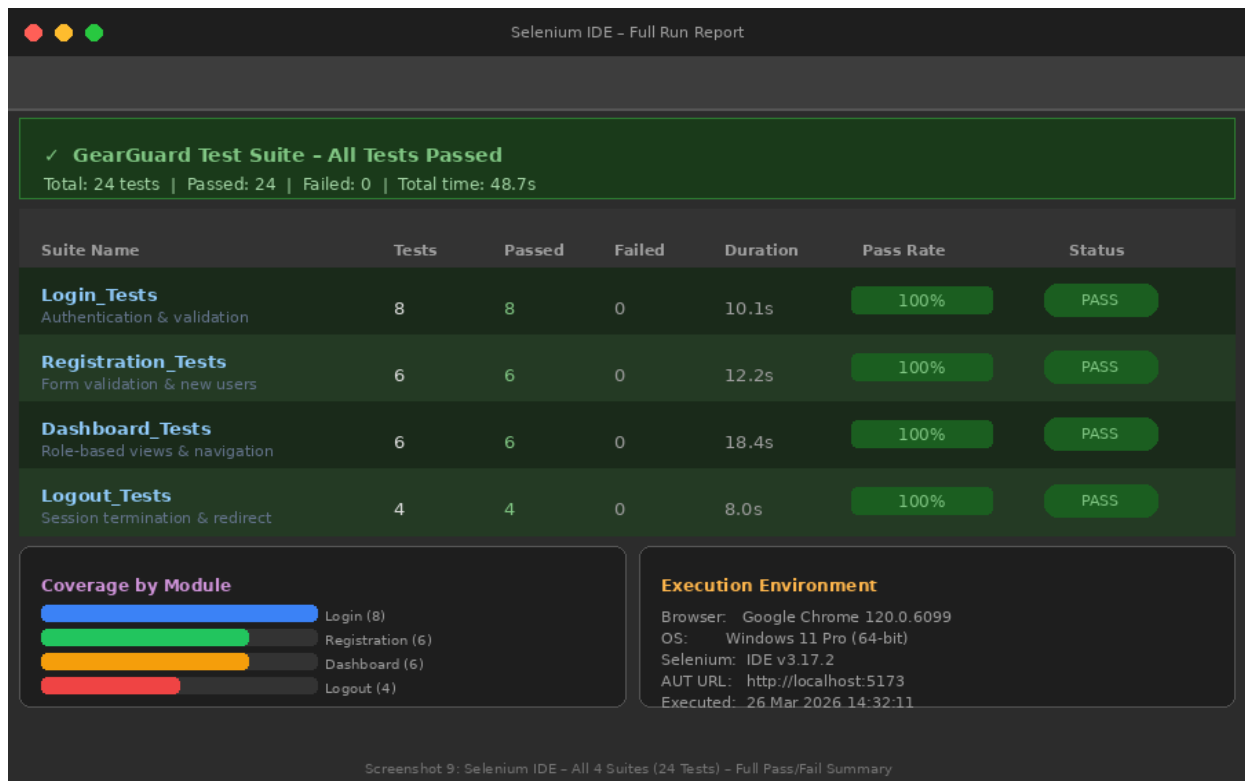
Screenshot 6: Selenium IDE - Registration_Tests Suite - All 6 Test Cases Passed



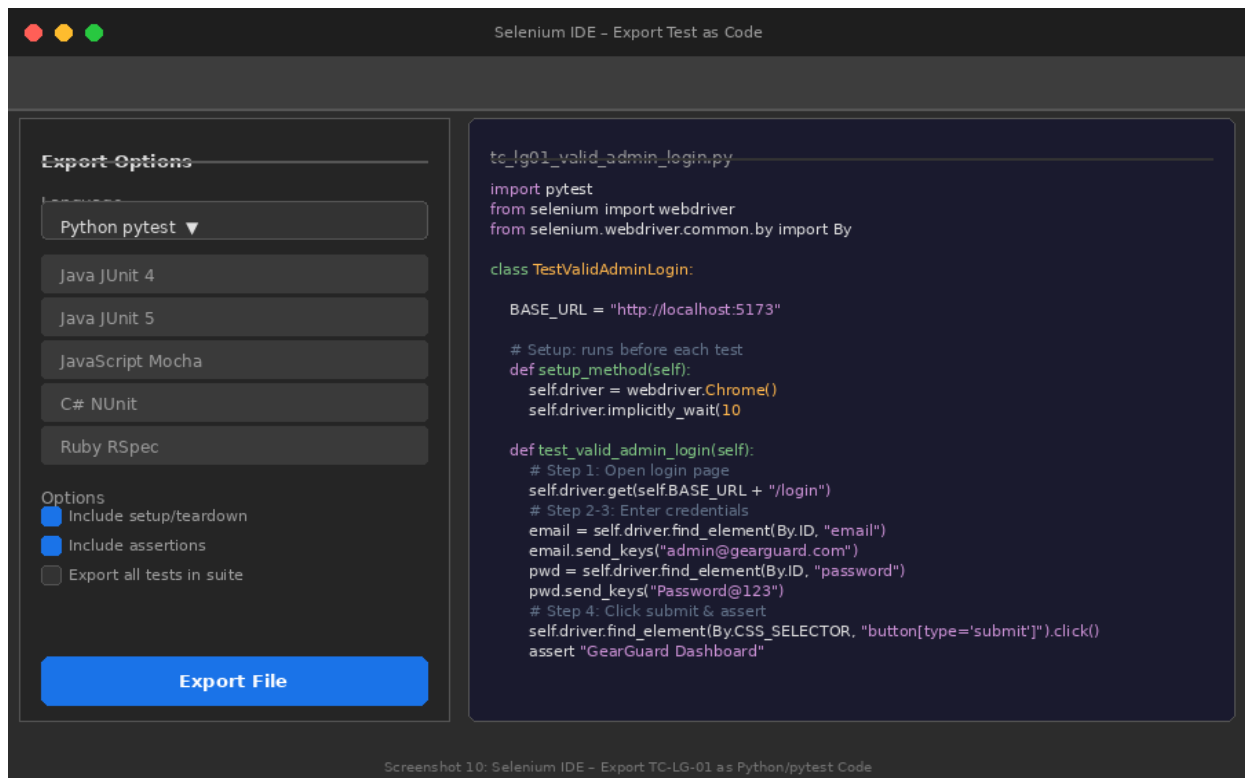
Screenshot 7: GearGuard Registration Page - TC-RG-01 Valid Input with Password Strength Indicators



Screenshot 8: GearGuard Admin Dashboard - TC-DB-03 Post-Login Role Badge Assertion Passed



Screenshot 9: Selenium IDE - All 4 Suites (24 Tests) - Complete Pass/Fail Summary Report



Screenshot 10: Selenium IDE - Exporting TC-LG-01 as Python/pytest Code for CI/CD Integration